Image Classification using Transfer Learning with CNN (ResNet18)

April 9, 2026

1 Introduction

Image classification is one of the fundamental problems in computer vision, where the goal is to assign a label to an input image from a predefined set of categories. Traditional machine learning approaches relied heavily on handcrafted feature extraction, which often failed to generalize across diverse datasets.

With the advancement of deep learning, Convolutional Neural Networks (CNNs) have become the standard approach for image-related tasks. However, training deep CNNs from scratch requires large datasets and significant computational resources.

To overcome this limitation, transfer learning is widely used. In transfer learning, a model pre-trained on a large dataset (such as ImageNet) is adapted to a new task. This project uses a pre-trained ResNet18 model and fine-tunes it on the CIFAR-10 dataset to achieve high classification accuracy with reduced training effort.

[54]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
from torchvision import datasets, transforms, models
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import numpy as np
```

[55]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cuda
```

2 Dataset Description

The dataset used in this project is CIFAR-10, which contains:

1) 60,000 images
2) 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
3) Each image is 32×32 pixels (resized during preprocessing)

The dataset is split into:

1) Training set (80%)
2) Validation set (20% of training)
3) Test set

[56]:
```
batch_size = 256
learning_rate = 0.001
num_epochs = 50
image_size = 224
```

3 Data Preprocessing and Augmentation

To improve generalization, several transformations were applied:

1) Resize and cropping
2) Random horizontal flip
3) Rotation
4) Color jittering
5) Normalization using ImageNet statistics

These augmentations help the model become robust to variations in input images.

[57]:
```
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomResizedCrop(image_size, scale=(0.8, 1.0)),    # zoom-like␣
 ↪effect
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),                         # rotation /␣
 ↪tilt
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
                           saturation=0.2, hue=0.05),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],              # ImageNet␣
 ↪normalization
                         std=[0.229, 0.224, 0.225])
])

test_transform = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])


###########
#find out the mean and std of the training data set
```

```
###########

#dataset = datasets.CIFAR10(root='./data', train=True, download=True,␣
↪transform=transform)
#loader = DataLoader(dataset, batch_size=500, shuffle=False)

#mean = 0.
#std = 0.
#total_images = 0

#for images, _ in loader:
#    batch_samples = images.size(0)  # batch size
#    images = images.view(batch_samples, images.size(1), -1)  # (B, C, H*W)

#    mean += images.mean(2).sum(0)
#    std += images.std(2).sum(0)
#    total_images += batch_samples

#mean /= total_images
#std /= total_images

#print("Mean:", mean)
#print("Std:", std)
```

[58]:
```
train_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=False,   # set True if not downloaded
    transform=train_transform
)

test_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=False,
    transform=test_transform
)

from torch.utils.data import random_split

train_size = int(0.8 * len(train_dataset))
val_size   = len(train_dataset) - train_size

train_dataset, val_dataset = random_split(train_dataset, [train_size, val_size])

val_dataset.dataset.transform = test_transform
```

```
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader  = DataLoader(test_dataset, batch_size=32, shuffle=False)

class_names = train_dataset.dataset.classes
num_classes = len(class_names)

print("Classes:", class_names)
print("Number of classes:", num_classes)
```

```
Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog',
'horse', 'ship', 'truck']
Number of classes: 10
```

4 Model Architecture

The project uses ResNet18, a deep CNN with residual connections.

Key features:

1) 18 layers deep
2) Residual (skip) connections to avoid vanishing gradients
3) Pre-trained on ImageNet

Modification: The final fully connected layer is replaced to output 10 classes instead of 1000

[59]:
```
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# Replace final fully connected layer
model.fc = nn.Linear(model.fc.in_features, num_classes)

model = model.to(device)
print(model)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (layer3): Sequential(
```

```
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
```

```
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=10, bias=True)
)
```

[60]:
```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Optional learning rate scheduler
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)
```

5 Training Setup

1) Loss Function: CrossEntropyLoss
2) Optimizer: Adam
3) Learning Rate: 0.001
4) Scheduler: StepLR (reduces learning rate every 5 epochs)
5) Epochs: 50

[61]:
```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    running_correct = 0
    total_samples = 0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        _, preds = torch.max(outputs, 1)
```

```
        running_correct += (preds == labels).sum().item()
        total_samples += labels.size(0)

    epoch_loss = running_loss / total_samples
    epoch_acc = running_correct / total_samples

    return epoch_loss, epoch_acc
```

[62]:
```
def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    running_correct = 0
    total_samples = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * images.size(0)
            _, preds = torch.max(outputs, 1)
            running_correct += (preds == labels).sum().item()
            total_samples += labels.size(0)

    epoch_loss = running_loss / total_samples
    epoch_acc = running_correct / total_samples

    return epoch_loss, epoch_acc
```

[63]:
```
import copy
best_val_acc = 0.0
best_model_wts = copy.deepcopy(model.state_dict())
tr_loss = []
tr_acc = []
va_loss = []
va_acc = []
ep = []

for epoch in range(num_epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion,␣
 ↪optimizer, device)
    val_loss, val_acc = evaluate(model, val_loader, criterion, device)

    scheduler.step()
```

```python
    print(f"Epoch [{epoch+1}/{num_epochs}]")
    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc*100:.2f}%")
    print(f"Val   Loss: {val_loss:.4f} | Val   Acc: {val_acc*100:.2f}%")
    print("-" * 50)

    tr_loss.append(train_loss)
    tr_acc.append(train_acc)
    va_loss.append(val_loss)
    va_acc.append(val_acc)
    ep.append(epoch+1)

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model_wts = copy.deepcopy(model.state_dict())

# Load best model
model.load_state_dict(best_model_wts)
print(f"Best Validation Accuracy: {best_val_acc*100:.2f}%")
```

```
Epoch [1/50]
Train Loss: 0.7241 | Train Acc: 75.21%
Val   Loss: 0.5560 | Val   Acc: 81.06%
--------------------------------------------------
Epoch [2/50]
Train Loss: 0.4303 | Train Acc: 85.34%
Val   Loss: 0.4047 | Val   Acc: 86.31%
--------------------------------------------------
Epoch [3/50]
Train Loss: 0.3098 | Train Acc: 89.37%
Val   Loss: 0.3715 | Val   Acc: 87.59%
--------------------------------------------------
Epoch [4/50]
Train Loss: 0.2356 | Train Acc: 91.77%
Val   Loss: 0.3353 | Val   Acc: 89.10%
--------------------------------------------------
Epoch [5/50]
Train Loss: 0.1705 | Train Acc: 94.08%
Val   Loss: 0.4280 | Val   Acc: 87.37%
--------------------------------------------------
Epoch [6/50]
Train Loss: 0.0482 | Train Acc: 98.56%
Val   Loss: 0.2599 | Val   Acc: 92.13%
--------------------------------------------------
Epoch [7/50]
Train Loss: 0.0168 | Train Acc: 99.64%
Val   Loss: 0.2654 | Val   Acc: 92.16%
--------------------------------------------------
```

```
Epoch [8/50]
Train Loss: 0.0085 | Train Acc: 99.83%
Val   Loss: 0.2798 | Val   Acc: 92.45%
--------------------------------------------------
Epoch [9/50]
Train Loss: 0.0052 | Train Acc: 99.91%
Val   Loss: 0.2981 | Val   Acc: 92.35%
--------------------------------------------------
Epoch [10/50]
Train Loss: 0.0031 | Train Acc: 99.95%
Val   Loss: 0.3118 | Val   Acc: 92.60%
--------------------------------------------------
Epoch [11/50]
Train Loss: 0.0023 | Train Acc: 99.96%
Val   Loss: 0.3105 | Val   Acc: 92.67%
--------------------------------------------------
Epoch [12/50]
Train Loss: 0.0017 | Train Acc: 99.98%
Val   Loss: 0.3111 | Val   Acc: 92.77%
--------------------------------------------------
Epoch [13/50]
Train Loss: 0.0014 | Train Acc: 99.98%
Val   Loss: 0.3096 | Val   Acc: 92.81%
--------------------------------------------------
Epoch [14/50]
Train Loss: 0.0013 | Train Acc: 99.99%
Val   Loss: 0.3132 | Val   Acc: 92.79%
--------------------------------------------------
Epoch [15/50]
Train Loss: 0.0011 | Train Acc: 99.99%
Val   Loss: 0.3078 | Val   Acc: 92.76%
--------------------------------------------------
```

6 Results and Analysis

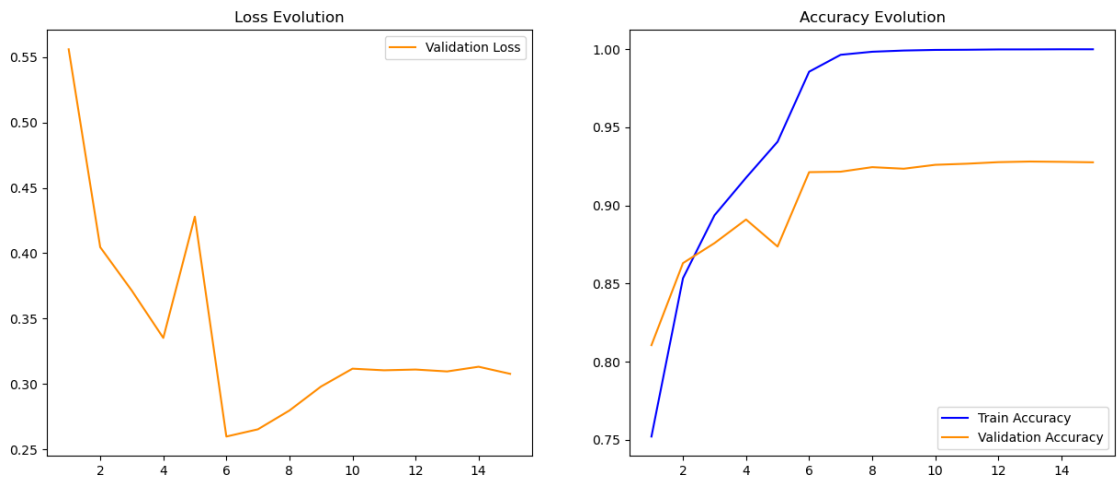
1) Training accuracy reaches nearly 100%
2) Validation accuracy stabilizes around 92–93%

[65]:
```
plt.figure(figsize=(15,6))

# Plotting the training and validation loss
plt.subplot(1, 2, 1)
#plt.plot(ep, tr_loss, label='Train Loss', color='blue')
plt.plot(ep, va_loss, label='Validation Loss', color='darkorange')
plt.legend()
plt.title('Loss Evolution')
```

```
# Plotting the training and validation accuracy
plt.subplot(1, 2, 2)
plt.plot(ep, tr_acc, label='Train Accuracy', color='blue')
plt.plot(ep, va_acc, label='Validation Accuracy', color='darkorange')
plt.legend()
plt.title('Accuracy Evolution')

plt.show()
```



[66]:
```
test_loss, test_acc = evaluate(model, test_loader, criterion, device)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc*100:.2f}%")

torch.save(model.state_dict(), "resnet18_best_model.pth")
print("Model saved as resnet18_best_model.pth")
```

```
Test Loss: 0.3166
Test Accuracy: 92.82%
Model saved as resnet18_best_model.pth
```

7 Prediction on External Images

The trained model was tested on real-world images:

1) Example 1: Cat → correctly classified as "cat"
2) Example 2: Frog → correctly classified as "frog"
3) Example 3: Accidental car → correctly classified as "Automobile"

This demonstrates the model's ability to generalize beyond the dataset.

[67]:
```
import requests
from PIL import Image
from io import BytesIO

url = "https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg"

headers = {
    "User-Agent": "Mozilla/5.0"
}

response = requests.get(url, headers=headers)

print("Status code:", response.status_code)

img = Image.open(BytesIO(response.content)).convert("RGB")
img.show()
```

```
Status code: 200

/usr/bin/xdg-open: 882: www-browser: not found
/usr/bin/xdg-open: 882: links2: not found
/usr/bin/xdg-open: 882: elinks: not found
/usr/bin/xdg-open: 882: links: not found
/usr/bin/xdg-open: 882: lynx: not found
/usr/bin/xdg-open: 882: w3m: not found
xdg-open: no method available for opening '/tmp/tmp5zmbveri.PNG'
```

[68]:
```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],
                         [0.229,0.224,0.225])
])

input_tensor = transform(img)
input_tensor = input_tensor.unsqueeze(0)
```

[69]:
```
input_tensor = input_tensor.to(device)

with torch.no_grad():
    outputs = model(input_tensor)
    _, predicted = torch.max(outputs, 1)

pred_class = predicted.item()
print("Predicted class index:", pred_class)
```

```
Predicted class index: 3
```

[70]:
```
class_names = ['airplane','automobile','bird','cat','deer',
               'dog','frog','horse','ship','truck']

print("Prediction:", class_names[pred_class])
```

```
Prediction: cat
```

[71]:
```
import matplotlib.pyplot as plt

plt.imshow(img)
plt.title(f"Prediction: {class_names[pred_class]}")
plt.axis("off")
plt.show()
```

Prediction: cat

[74]:
```
url = "https://cdn.britannica.com/72/45872-050-10F6A603/Green-frog.jpg"

headers = {
    "User-Agent": "Mozilla/5.0"
}

response = requests.get(url, headers=headers)
```

```python
img = Image.open(BytesIO(response.content)).convert("RGB")
img.show()

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],
                         [0.229,0.224,0.225])
])

input_tensor = transform(img)
input_tensor = input_tensor.unsqueeze(0)

input_tensor = input_tensor.to(device)

with torch.no_grad():
    outputs = model(input_tensor)
    _, predicted = torch.max(outputs, 1)

pred_class = predicted.item()

plt.imshow(img)
plt.title(f"Prediction: {class_names[pred_class]}")
plt.axis("off")
plt.show()
```

```
/usr/bin/xdg-open: 882: www-browser: not found
/usr/bin/xdg-open: 882: links2: not found
/usr/bin/xdg-open: 882: elinks: not found
/usr/bin/xdg-open: 882: links: not found
/usr/bin/xdg-open: 882: lynx: not found
/usr/bin/xdg-open: 882: w3m: not found
xdg-open: no method available for opening '/tmp/tmpnncvwa1e.PNG'
```

Prediction: frog

[30]:
```
img = Image.open('car.jpg').convert("RGB")
img.show()

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],
                         [0.229,0.224,0.225])
])

input_tensor = transform(img)
input_tensor = input_tensor.unsqueeze(0)

input_tensor = input_tensor.to(device)

with torch.no_grad():
    outputs = model(input_tensor)
    _, predicted = torch.max(outputs, 1)

pred_class = predicted.item()

plt.imshow(img)
plt.title(f"Prediction: {class_names[pred_class]}")
```

```
plt.axis("off")
plt.show()
```

```
/usr/bin/xdg-open: 882: www-browser: not found
/usr/bin/xdg-open: 882: links2: not found
/usr/bin/xdg-open: 882: elinks: not found
/usr/bin/xdg-open: 882: links: not found
/usr/bin/xdg-open: 882: lynx: not found
/usr/bin/xdg-open: 882: w3m: not found
xdg-open: no method available for opening '/tmp/tmp8hsiuspo.PNG'
```

Prediction: automobile